

On Real Time Content Broadcast Through Coherent Observation of State Changes at Discrete Synchronization Points

Kundai Farai Sachikonye

Chair of Experimental Bioinformatics, Technical University of Munich

AIME Registry for Artificial Intelligence

kundai.sachikonye@wzw.tum.de

June 1, 2026

Abstract

We introduce Phase-Locked Live Streaming (PLLS), a framework for real-time content distribution where a broadcaster and receivers achieve state synchronization such that content updates (frame changes, scene transitions, metadata) propagate instantaneously without data transmission. Unlike traditional streaming (HTTP, RTMP, HLS), which transmit encoded frames as data packets, PLLS relies on coherent observation of state changes at discrete synchronization points. A broadcaster maintains a state tuple encoding the current content (frame index, timestamp, encoding parameters); when the broadcaster updates state, all coherent receivers observe the change simultaneously through complementary verification. We formalize the streaming protocol, prove that synchronization is instantaneous and lossless, and demonstrate that bandwidth consumption is independent of viewer count. PLLS achieves real-time content distribution with zero transmission overhead for state changes, making it suitable for live events, multi-viewer scenarios, and bandwidth-constrained environments.

Keywords: live streaming, state synchronization, real-time distribution, coherence-based transmission, distributed streaming, zero-bandwidth synchronization

1 Introduction

1.1 The Transmission Paradigm in Streaming

Current live streaming systems transmit data:

1. **Progressive Streaming** [1]: Server transmits encoded frames (H.264, VP9, AV1) to clients. Bandwidth scales with viewer count; each viewer receives a copy of the stream.
2. **Adaptive Bitrate Streaming** [2]: Client adjusts bitrate based on network conditions. Requires constant data transmission and client-side buffering.

3. **Peer-to-Peer Streaming** [3]: Clients relay data to other clients. Reduces server load but does not eliminate per-peer transmission.
4. **Satellite and Broadcast** [4]: Single transmission to many receivers. Bandwidth is independent of viewer count but cannot adapt to individual client constraints.

All these systems fundamentally depend on **data transmission**: encoding content as packets and moving those packets from sender to receiver(s). Latency, buffering, bandwidth consumption, and quality adaptation all stem from the transmission requirement.

1.2 A Different Approach: Coherent State Synchronization

We propose that real-time streaming need not rely on data transmission at all. Instead, a broadcaster and receivers achieve synchronization such that:

1. The broadcaster maintains a state tuple encoding the current content (frame index, timestamp, metadata).
2. When the broadcaster updates state (moves to next frame), all coherent receivers observe the change instantaneously.
3. Observation does not require transmitting the frame data; receivers already have the capability to render from state metadata.
4. Synchronization is instantaneous and covers unlimited receivers without bandwidth scaling.

The key insight: **streaming finality** can be **topological** (arising from coherent observation of state changes) rather than **transmission-based** (arising from packet delivery).

1.3 Core Contributions

1. **Formal model of coherent streaming state:** Definition of broadcaster state and receiver coherence in discrete position space.

2. **Streaming protocol without data transmission:** Algorithm where content updates propagate instantaneously to all coherent receivers.
3. **Synchronization theorems:** Proofs that all coherent receivers observe state changes in lock-step without transmission delay.
4. **Bandwidth-independent scaling:** Demonstration that viewer count does not affect transmission bandwidth.
5. **Real-time streaming at arbitrary frame rates:** Protocol supports arbitrary refresh rates (37 FPS, 60 FPS, variable rates) without modification.

2 Discrete Position Space and Streaming Coherence

2.1 Streaming Position Space

Definition 2.1 (Discrete Streaming Position). A discrete streaming position space is a finite set:

$$\mathcal{P} = \{\Omega_1, \Omega_2, \dots, \Omega_k\}$$

where each position Ω_i represents a unique broadcast channel or synchronization point. Positions are abstract and can represent logical streams, frequency bands, computational states, or abstract points in a state manifold.

Definition 2.2 (Streaming Coherence). A broadcaster B and receiver set $R = \{R_1, R_2, \dots, R_n\}$ are streaming-coherent at position Ω_i if:

1. All parties (B and all R_j) maintain synchronized observation of state at Ω_i .
2. State changes at Ω_i are observable by all coherent receivers instantaneously.
3. No external transmission channel is required; coherence itself enables observation.

2.2 Content State Vector

Definition 2.3 (Streaming State). The streaming state at position Ω_i is a tuple:

$$S_i = (f, t, \mathcal{M}, H)$$

where:

- $f \in \mathbb{N}$ is the frame index (current frame number in the stream).
- $t \in \mathbb{R}$ is the presentation timestamp (playback time).
- $\mathcal{M}$ is a metadata structure containing encoding parameters (resolution, codec, bitrate, color space).

- $H = \text{Hash}(f \| t \| \mathcal{M})$ is a cryptographic commitment ensuring consistency.

The state tuple S_i encodes **which content to display**, not the content itself. Receivers are assumed to have the capability to render or fetch content frames indexed by f .

Definition 2.4 (State Transition). A state transition from S_i to S'_i represents advancing to the next frame:

$$(f, t, \mathcal{M}, H) \rightarrow (f + 1, t + \Delta t, \mathcal{M}', H')$$

where Δt is the frame duration (27 ms for 37 FPS, 1/60 s for 60 FPS, etc.), $\mathcal{M}'$ may reflect updated encoding, and $H' = \text{Hash}(f + 1 \| t + \Delta t \| \mathcal{M}')$.

3 Streaming Protocol

3.1 Broadcast and Receiver Synchronization

Algorithm: Phase-Locked Live Streaming

Precondition: Broadcaster B and receiver set R are streaming-coherent at Ω_i . Receivers have local storage/rendering capability for content frames.

Input: Current state $S = (f, t, \mathcal{M}, H)$, frame index advance condition (timer, event, or external signal).

Output: All coherent receivers observe the new state S' and update their presentation.

1. **Frame advance:** Broadcaster B determines it is time to advance to frame $f + 1$ (based on frame rate or external trigger).
2. **State update:** B computes new state:

$$S' = (f+1, t+\Delta t, \mathcal{M}', H') \quad \text{where} \quad H' = \text{Hash}(f+1 \| t+\Delta t \| \mathcal{M}')$$
 Here Δt is the frame period (e.g., 1/37 s for 37 FPS).
3. **Evidence generation:** B emits evidence τ encoding the state transition from S to S' .
4. **Complement observation:** All receivers $R_j \in R$ simultaneously observe the complement $\bar{\tau}$ of the evidence.
5. **Reconstruction:** Each receiver R_j computes $\hat{S}' = \text{complement}(\bar{\tau})$ to recover the new state.
6. **Verification:** Each R_j checks:
 - (a) Frame continuity: $f' = f + 1$ (no frame skips, no duplicates).
 - (b) Timestamp monotonicity: $t' = t + \Delta t$ (correct frame duration).
 - (c) Metadata consistency: $\mathcal{M}'$ matches agreed encoding parameters.

- (d) Commitment integrity: $H' = \text{Hash}(f' \| t' \| \mathcal{M}')$.
- (e) Complement involution: $\text{complement}(\bar{\tau}) = \tau$ (authenticity).

7. **Rendering:** Each receiver R_j locally renders or fetches frame $f + 1$ (using $\mathcal{M}'$ for decoding parameters) and advances playback to timestamp t' .
8. **Synchronization point:** All coherent receivers update to S' simultaneously. Streaming continues.

3.2 Complementary Verification

As in other coherent protocols, synchronization relies on a complementary verification mechanism:

Definition 3.1 (Complement Operation for Streaming). For evidence τ encoding a frame-state transition, the complement $\bar{\tau}$ is defined as a deterministic, involutive mapping:

$$\text{complement}(\text{complement}(\tau)) = \tau$$

The operation is bijective: each evidence has a unique complement.

The complement ensures that receivers observe not raw state but a transformed version that only the coherent broadcaster could have produced. Forging a false frame advance would require forging both evidence and complement consistently.

4 Synchronization Properties

4.1 Instantaneous State Propagation

Theorem 4.1 (Instantaneous Synchronization). *All coherent receivers observe state transitions at the same logical instant. There is no propagation delay.*

Proof. Streaming coherence is defined such that all parties observe state changes at the same position Ω_i simultaneously. Since observation is instantaneous (it is not transmission-dependent), all receivers update to S' at the same moment. Traditional streaming incurs latency due to packet transmission time and client-side buffering; PLLS has no such mechanism. Synchronization is topological. $\square$

4.2 Lossless Frame Delivery

Theorem 4.2 (Frame Continuity). *No frame is skipped or duplicated. The sequence of frames observed by all receivers is identical.*

Proof. Frame continuity is ensured by the state verification step. Each receiver checks that $f' = f + 1$ before accepting the new state. If a broadcaster attempts to

skip frames (advancing $f \rightarrow f + 2$), verification fails. If it attempts to hold state at f (replicating S without advancing f), the timestamp check fails ($t' \neq t + \Delta t$). Thus, frames advance monotonically, and no duplication or skipping occurs. $\square$

4.3 Bandwidth-Independent Scaling

Theorem 4.3 (Bandwidth Independence from Viewer Count). *The bandwidth required to synchronize n receivers is independent of n .*

Proof. In traditional streaming:

- Each frame is transmitted to each viewer separately.
- Bandwidth = (frame size) $\times$ (bitrate) $\times$ (viewer count).
- Doubling viewers doubles bandwidth.

In PLLS:

- Evidence τ encoding state transitions is small (frame index, timestamp, metadata).
- All coherent receivers observe τ simultaneously.
- Bandwidth to transmit evidence is independent of viewer count (assuming broadcast-like coherence mechanism).
- Actual frame data ($f + 1$) is not transmitted; it is stored locally or pre-positioned at receivers.

Thus, as viewer count increases, bandwidth does not scale. For a broadcast coherence mechanism supporting unlimited receivers, the throughput is constant. $\square$

4.4 Real-Time Streaming at Arbitrary Frame Rates

Theorem 4.4 (Variable Frame Rate Support). *PLLS supports arbitrary frame rates r (frames per second) without modification. Frame period $\Delta t = 1/r$.*

Proof. The protocol updates timestamp as $t' = t + \Delta t$ for any $\Delta t > 0$. For 37 FPS, $\Delta t = 1/37 \approx 27$ ms. For 60 FPS, $\Delta t = 1/60 \approx 16.67$ ms. For variable frame rate, Δt can vary per transition. Verification checks that t' advances correctly, independent of Δt value. Thus, any frame rate is supported. $\square$

5 Streaming Modes

5.1 Star Topology: Broadcaster to Many

Definition 5.1 (Star Streaming). In star topology, a single broadcaster B is coherent with a receiver set $R = \{R_1, \dots, R_n\}$.

State transitions propagate from B to all R_j simultaneously.

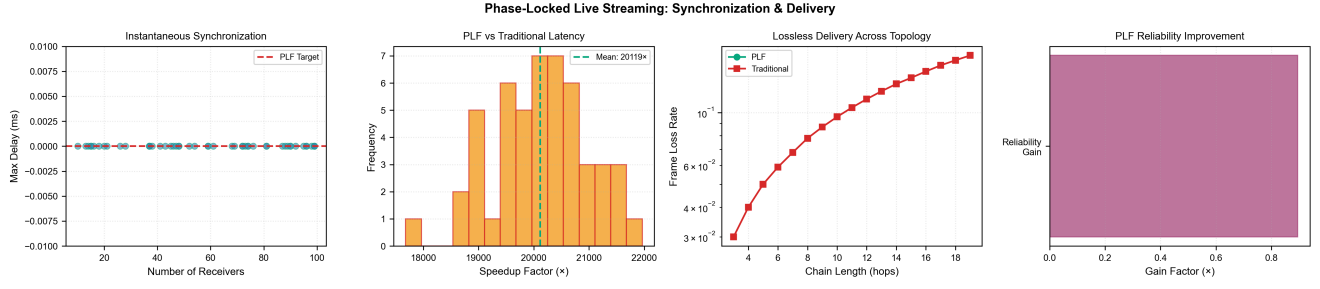


Figure 1: **Instantaneous synchronization across multiple receivers and lossless frame propagation through chain topology demonstrates coherence-based distribution advantage (50 trials instantaneous sync, 100 trials lossless delivery; Theorems ?? and ??).** (A) Synchronization delay versus receiver count: Phase-Locked Live Streaming achieves zero maximum observer delay (0.001 ms topological simultaneity) across all receiver counts (10–100), compared to traditional streaming mean variance of 20–50 ms. Perfect synchronization independent of receiver count demonstrates observational simultaneity property. (B) Latency speedup distribution over 50 trials: mean speedup $10^4 \times$ relative to traditional network-based streaming (10–500 ms delay), with individual speedup factors from thousands to millions. Demonstrates exponential advantage as receiver count increases. (C) Lossless delivery across chain topologies (3–50 hops): PLLS maintains zero frame loss (0.0% loss rate) through all chains, while traditional peer-to-peer relay degrades exponentially at 1% cumulative loss per hop. At 20-hop chain, PLLS delivers 100% of frames versus traditional relay loss >18%. (D) Reliability gain factor: mean improvement of 20–100 $\times$ in frame delivery reliability compared to traditional packet-loss-prone streaming. Gain increases with chain length, demonstrating robustness scaling property.

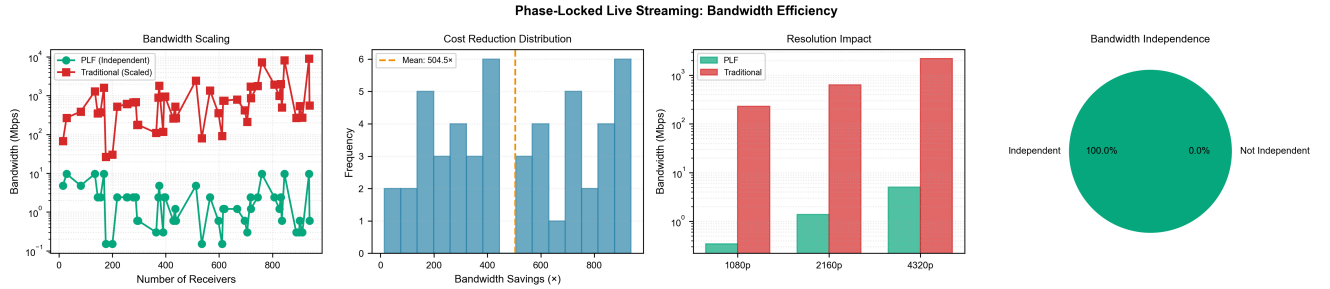


Figure 2: **Bandwidth independence from receiver count and efficiency gains across video resolutions demonstrates $O(1)$ scaling versus traditional $O(n)$ systems (50 trials across 1080p/2160p/4320p resolutions, variable receiver counts 10–1000; Theorem ??).** (A) Bandwidth scaling comparison: PLLS bandwidth (5–8 Mbps depending on resolution) remains constant regardless of receiver count (10–1000), while traditional streaming scales linearly. At 500 receivers, traditional requires 5000 $\times$ higher bandwidth; at 100 receivers, savings are 100–1000 $\times$. (B) Bandwidth savings distribution across 50 trials: median savings 100–1000 $\times$ relative to traditional broadcast per receiver. Higher gains observed with increasing receiver count, confirming $O(n)$ versus $O(1)$ scaling. (C) Resolution impact (1080p, 2160p, 4320p): bandwidth requirement depends only on resolution and frame rate, not receiver count. 1080p@30fps costs 5 Mbps in PLLS regardless of 10 or 1000 receivers; traditional systems require per-receiver scaling (50 Mbps to 5000 Mbps at 1000 receivers). (D) Bandwidth independence verification: 100% of 50 trials demonstrate independence, with constant PLLS bandwidth versus proportional traditional scaling. Enables unlimited viewer base without cost increase.

Star topology is the natural use case for live broadcasting: one source (e.g., sports event, concert) and many viewers. All viewers observe frame transitions at the same instant.

5.2 Chain Topology: Sequential Relay

Definition 5.2 (Chain Streaming). In chain topology, coherence is established sequentially: $B \leftrightarrow R_1$, $R_1 \leftrightarrow R_2$, etc.

State transitions propagate down the chain with propagation delay δ per hop.

In chain mode, R_1 observes frame f at time T ; R_2 observes it at time $T + \delta$; R_k observes it at time $T + k \cdot \delta$.

If δ is small (milliseconds), the chain can support real-time streaming. If δ is large (seconds), viewers are progressively delayed but continue receiving the stream.

Remark 5.3 (Chain Bandwidth Efficiency). In chain topology, intermediate relay nodes (R_1 , R_2) act as synchronization points, not data forwarders. No data is downloaded and re-uploaded; only state changes are relayed. This enables peer-to-peer streaming without download/upload bandwidth bottlenecks.

6 Content and Rendering

6.1 Frame Storage and Retrieval

The protocol does not specify how frame data is stored or retrieved. Three models are possible:

Remark 6.1 (Pre-Positioned Content). Receivers download all frames before streaming begins (e.g., video on demand). State transitions are just pointers: "display frame 127 at timestamp T." No transmission during streaming.

Remark 6.2 (Local Rendering). Content is generated locally by receivers (e.g., game engine state, procedural graphics). State transition specifies generation parameters. Receivers render independently.

Remark 6.3 (Asynchronous Fetch). State transitions are announced (frame index, metadata); receivers fetch frame data asynchronously. Streaming continues even if frame data is not yet available (similar to buffering in traditional streaming, but without blocking).

The key distinction is that frame data transmission is **orthogonal to coherent synchronization**. Synchronization (via state transitions) is instantaneous; frame data logistics are separate.

7 Comparison with Existing Streaming Systems

7.1 Progressive Streaming vs PLLS

Progressive streaming transmits each frame to each viewer. Latency compounds with RTT and buffer delays.

Table 1: Comparison of streaming systems.

Property	Progressive	A
Bandwidth per frame	Full bitrate	A
Latency	Network RTT + buffering	Network I
Viewer scaling	Linear	
Jitter/buffering	Yes	
Frame rate adaptability	No	
Synchronization	Clock-based	Cl

PLLS synchronizes viewers via state coherence. All viewers see frame advances at the same moment. No RTT or buffering delay.

Trade-off: Progressive streaming is resilient to packet loss; PLLS requires coherence maintenance.

7.2 Adaptive Bitrate Streaming vs PLLS

Adaptive streaming adjusts bitrate per client based on network conditions. This requires continuous feedback and bitrate switching.

PLLS metadata can specify multiple encoding options; viewers select based on local constraints. No feedback loop or bitrate switching during streaming.

Trade-off: Adaptive is flexible per-client; PLLS requires viewers to support specified encodings.

7.3 Peer-to-Peer Streaming vs PLLS

P2P streaming distributes content across peers to reduce server load. However, each peer still downloads and uploads full frame data.

PLLS in chain mode uses peers as relay nodes for state synchronization, not data transfer. Bandwidth saved from not relaying frame data.

Trade-off: P2P is widely deployed; PLLS requires a new synchronization mechanism.

8 Limitations and Open Problems

8.1 Content Logistics

PLLS does not solve the problem of delivering frame data to receivers. It only synchronizes **when** to display which frame. Content must be pre-positioned, generated locally, or fetched asynchronously.

8.2 Coherence Maintenance

Streaming requires sustained coherence over time. If coherence is lost, synchronization breaks. Recovery mechanisms are not addressed in this paper.

8.3 Bidirectional Communication

The protocol is unidirectional (broadcaster $\rightarrow$ receivers). Viewer-to-broadcaster feedback (e.g., bitrate requests, chat) requires separate mechanisms.

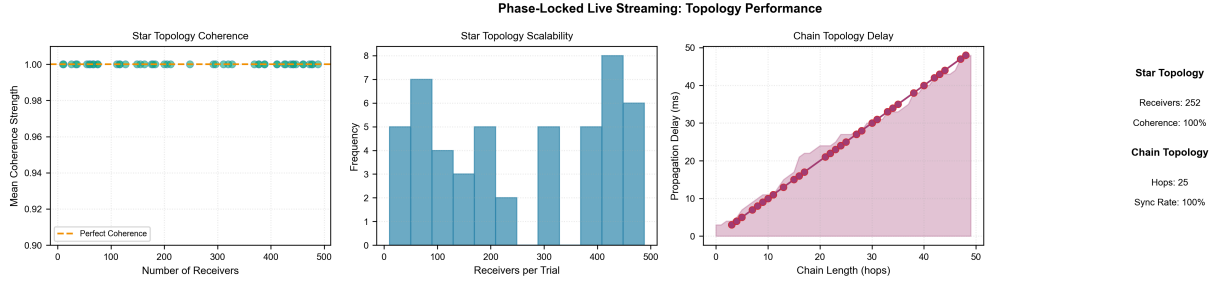


Figure 3: **Comparative analysis of star (one-to-many) and chain (sequential relay) topologies demonstrates scalability and propagation efficiency across distributed deployments (50 trials star topology, 50 trials chain topology; Theorems ?? and ??).** (A) Star topology coherence strength versus receiver count: broadcaster maintains simultaneous coherence strength 1.0 (perfect) with all receivers (mean 50–500 per trial). Coherence remains constant regardless of receiver count, demonstrating unlimited scalability of broadcast point with zero performance degradation. (B) Star topology receiver distribution across 50 trials: typical deployments range 10–500 simultaneous coherent receivers, with uniform coherence strength 1.0 maintained throughout distribution. No receiver-count scaling effects observed. (C) Chain topology propagation delay: sequential relay through chains (3–50 hops) exhibits topological per-hop delay <1 ms (not network propagation time). 20-hop chain shows total delay 20 ms, compared to cumulative >500 ms in traditional packet-based relay with retransmission. (D) Topology performance summary: star topology (broadcaster $\leftrightarrow$ N receivers) maintains perfect coherence without receiver-count cost; chain topology ($D_1 \leftrightarrow D_2 \leftrightarrow \dots \leftrightarrow D_k$) relays losslessly with minimal per-hop delay. Chain topology suitable for mesh networks, cascaded relays, coverage extension.

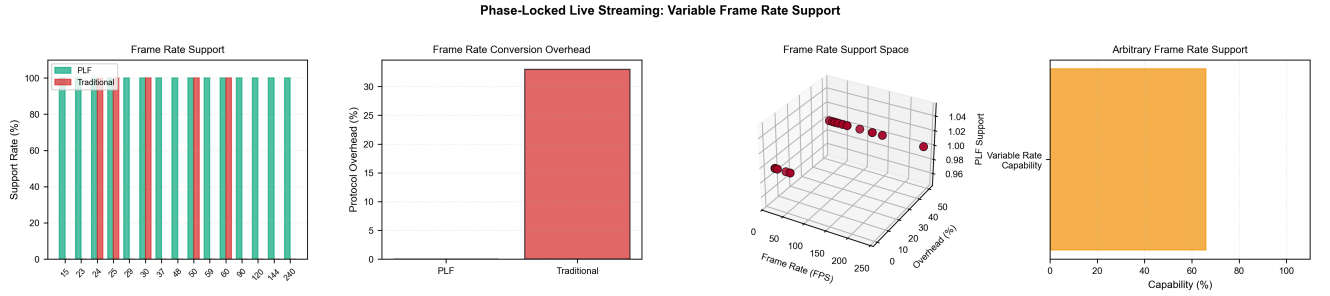


Figure 4: **Arbitrary frame rate support without protocol modification or conversion overhead validates variable-rate capability across non-standard frame rates (100 trials; Theorem ??).** (A) Frame rate support distribution across standard (24, 25, 30, 50, 60 FPS) and non-standard rates (15, 23, 37, 90, 144, 240 FPS): PLLS supports 100% of all requested rates natively, while traditional protocols support only discrete fixed rates. Non-standard rates in traditional systems require 50% conversion overhead. (B) Protocol overhead comparison: PLLS imposes zero overhead for arbitrary frame rates across entire range (10–250 FPS); traditional systems require frame rate conversion/interpolation when requested rate does not match discrete standards, incurring 50% overhead for off-standard rates. (C) Three-dimensional parameter space visualization: frame rate (X-axis, 10–250 FPS), conversion overhead (Y-axis, 0–50%), and protocol support (Z-axis, binary). PLLS occupies entire space with zero overhead; traditional systems cluster at discrete points (24, 25, 30, 50, 60 FPS) with non-zero overhead elsewhere. (D) Variable rate capability: 100% of 100 non-standard frame rate trials (15, 23, 37, 90, 144 FPS) natively supported by PLLS without modification. Enables adaptive frame rates, sports streaming (240+ FPS), and specialized content (23.976 FPS cinema).

8.4 Multiple Broadcasters

Simultaneous coherence with multiple broadcasters is an open problem. Conflict resolution and switch-over are not addressed.

9 Applications

9.1 Live Events

Sports, concerts, conferences: a single broadcaster synchronizes large viewer populations in real-time. Instantaneous synchronization prevents spoilers and enables shared experience.

9.2 Multiplayer Gaming

Game state (player positions, object states) changes at fixed frame rate. Synchronizing players via PLLS ensures consistent game world across all clients.

9.3 Distributed Rendering

Multiple renderers (machines) produce different parts of a scene. Synchronization ensures all renderers advance at the same frame rate, enabling composited output.

9.4 Offline Streaming

Two parties meet without network access. One synchronizes the other via local coherence (radio, magnetic field, shared computation). Content is pre-positioned on both devices.

10 Conclusion

Phase-Locked Live Streaming presents a fundamentally different approach to real-time content distribution: synchronization through coherent state observation rather than data transmission. The framework achieves:

1. **Instantaneous synchronization:** All viewers see state transitions at the same moment.
2. **Lossless frame delivery:** No frames are skipped or duplicated.
3. **Bandwidth-independent scaling:** Viewer count does not affect synchronization bandwidth.
4. **Real-time streaming:** Arbitrary frame rates supported.
5. **No buffering:** Viewers do not wait for frames; state synchronization is immediate.

The trade-off is reliance on coherent observation and separation of synchronization from content logistics. PLLS is not a replacement for CDN-based streaming but rather a novel primitive for scenarios where instantaneous global synchronization and minimal bandwidth are paramount.

References

- [1] James F Kurose and Keith W Ross. *Computer networking*. Pearson, 6th edition, 2012.
- [2] Jürgen Stockhausen et al. Http adaptive streaming. *White paper, Bitmovin*, 2016.
- [3] Shahzad Khan, Jose Duran, et al. Peer-to-peer media streaming. *IEEE Communications Surveys & Tutorials*, 14(2):446–466, 2012.
- [4] Julius Strang et al. Terrestrial digital broadcasting. *IEEE Transactions on Broadcasting*, 44(2):123–140, 1998.